

Introduction to Java Programming

Introduction

- Java (earlier named Oak) was designed by James Gosling and his colleagues at Sun Microsystems in early 90s and first released in 1995
- Major principles
 - Object-oriented
 - Portable / Architecture-neutral
 - Simple and familiar
- Use
 - Initially designed for embedded applications
 - Popularized by the Web phenomenon
 - Highly successful in enterprise application development as well as mobile application development

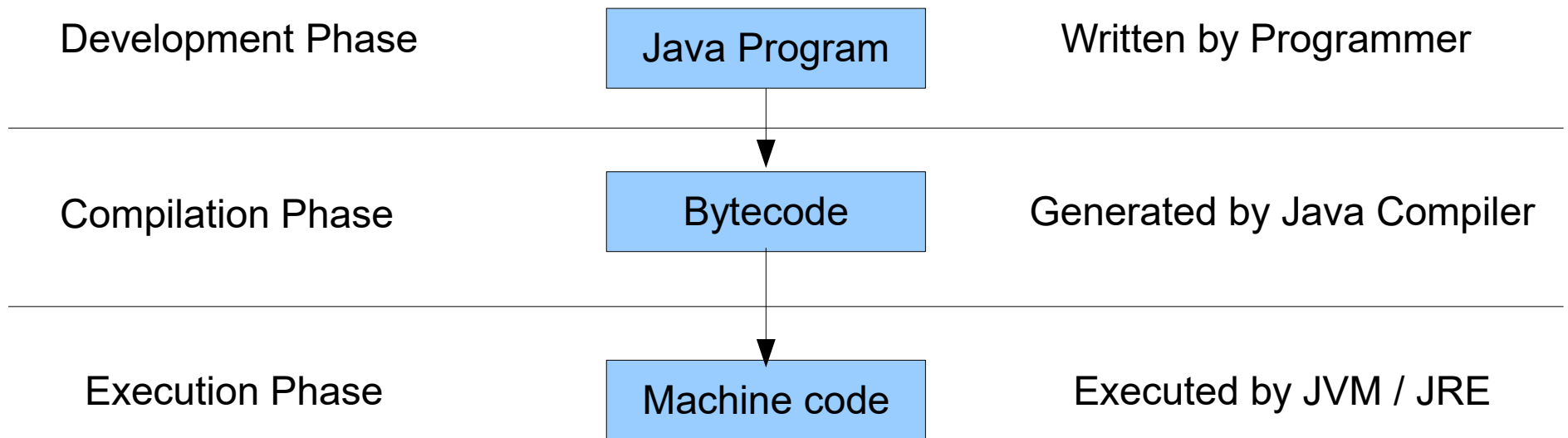
Java vs C++

- Java has:
 - same syntax as C++ with minor variations
 - Complete support for Object-oriented programming
 - Garbage collection
- Java does not have:
 - Pointers
 - Instead **references** exist for manipulating objects
 - Multiple inheritance
 - Instead multiple **interfaces** can be implemented
 - Templates
 - Instead **generics** are used for generic programming using principles of polymorphism

Other differences

- Almost everything is an object
 - Except for primitives (int, char, boolean, float, long, float, double)
 - Every Class inherits from Object
- All methods are virtual i.e. dynamically bound
- Method arguments are passed by value
 - For objects, references are passed as a copy
- Everything shall be a class
 - No structs, no stand-alone functions
- No separation between header and implementation
- No operator overloading
- Comes with many pre-defined libraries
- Slower than C++ but still efficient
 - Adequate for all scenarios except for time critical applications

Portability



Hello World Program

Development Phase	<pre>helloworldprogram/Main.java package helloworldprogram; class Main { public static void main(String args[]){ System.out.println("Hello world"); } }</pre>
-------------------	--

Compilation Phase	<pre>javac helloworldprogram/Main.java</pre>
-------------------	--

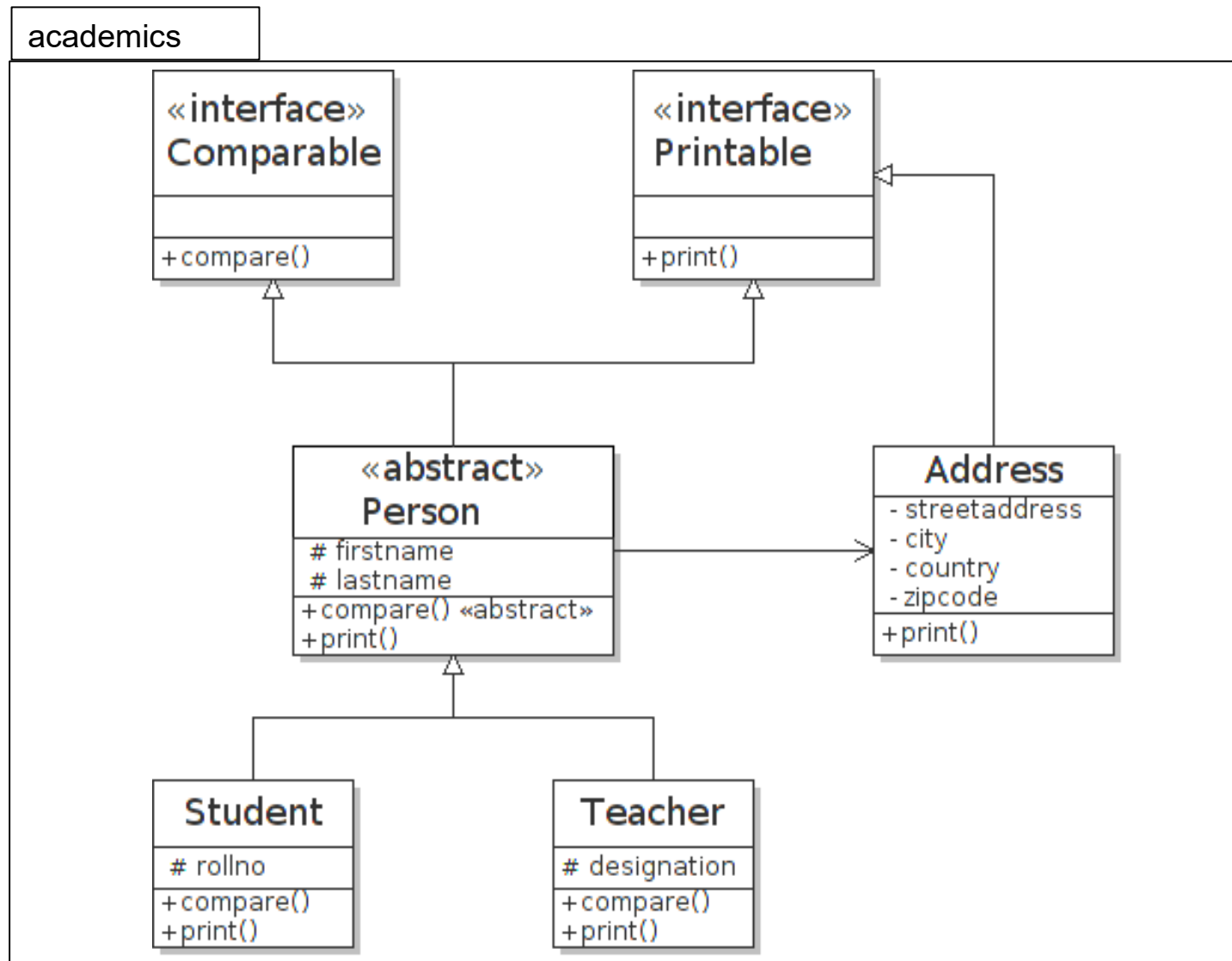
Execution Phase	<pre>java helloworldprogram.Main</pre>
-----------------	--

Object-oriented Programming

Overview

- Concepts
 - Object
 - Object associations
 - Inheritance & Polymorphism
 - Exception Handling
- Constructs
 - package, import
 - class
 - abstract class
 - interface
 - new
 - try, catch, throw, throws

Example



Interface

- Interface

- Purely virtual / abstract
- Only contains method signatures
- Java 8 allows default methods for evolving interfaces
- Used to define a public interface for a class
- Class implements an interface
- Class can implement multiple interfaces leading to multiple type inheritance

Comparable.java

```
package academics;

public interface Comparable {
    int compare(Comparable obj) throws NotComparableException;
}
```

Printable.java

```
package academics;

public interface Printable {
    void print();
}
```

Abstract class

- Abstract class
 - cannot be instantiated
 - at least one abstract method
 - abstract keyword required with class and method
 - may implement interfaces
 - not providing the implementation of an interface method automatically makes the class abstract

Person.java

```
package academics;

public abstract class Person implements Comparable, Printable{

    protected String firstName;
    protected String lastName;
    protected Address address;

    public Person(){

    }

    public Person(String fname,String lname){
        firstName = fname;
        lastName = lname;
    }

    public String getName(){
        return firstName + " " + lastName;
    }

    public void print(){
        System.out.println(getName());

        try{
            address.print();
        }
        catch (NullPointerException ex){
            System.out.println("No address recorded");
        }

        System.out.println("---");
    }

    public abstract int compare(Comparable obj) throws NotComparableException;
}
```

Address.java

```
package academics;

public class Address implements Printable{

    private String streetAddress;
    private String city;
    private String country;
    private String zipCode;

    @Override
    public void print() {
        System.out.println(streetAddress + "\n" +
                           city          + "\n" +
                           country       + "\n" +
                           zipCode      + "\n" );
    }
}
```

Inheritance

- Inheritance
 - specified through **extends** keyword
 - only single inheritance allowed – cannot inherit from multiple classes
 - Overriding a method hides the parent implementation but can be accessed through **super** keyword
 - super may also be used in constructor for initialization of parent – must be the first statement in constructor
 - all classes extend from Object by default

Student.java

```
package academics;

public class Student extends Person{

    protected String rollNumber;

    public Student(String rollNum,String fName,String lName){
        super(fName,lName);
        rollNumber = rollNum;
    }

    public int compare(Comparable obj)
        throws NotComparableException{
        if (obj instanceof Student){
            Student student = (Student) obj;

            return this.getName().compareTo(student.getName());
        }
        else throw new NotComparableException();
    }

    public void print() {
        System.out.println(rollNumber);
        super.print();
    }
}
```

Teacher.java

```
package academics;

public class Teacher extends Person{

    protected String designation;

    public Teacher(String designation,String fName,String lName){
        super(fName,lName);
        this.designation = designation;
    }

    @Override
    public int compare(Comparable obj)
        throws NotComparableException{
        if (obj instanceof Teacher){
            Teacher teacher = (Teacher) obj;

            return this.getName().compareTo(teacher.getName());
        }
        else throw new NotComparableException();
    }

    @Override
    public void print() {
        System.out.println(designation);
        super.print();
    }
}
```

Main.java

```
public class Main {

    public static void main(String[] args) {
        Address addr = new Address("FAST-NU, Block-B, Faisal Town",
                                    "Lahore", "Pakistan");

        Person p1 = new Student("20-1234", "Ahmed", addr);
        Person p2 = new Student("20-1235", "Ali", addr);
        Person p3 = new Teacher("Assistant Professor", "Farooq", addr);

        p1.print();
        p2.print();
        p3.print();

        try{
            System.out.println(p1.compare(p1));
            System.out.println(p1.compare(p2));
            System.out.println(p2.compare(p1));
            System.out.println(p2.compare(p3));
        }
        catch (Exception ex){
            System.out.println(ex.getMessage());
        }

    }

}
```

Output

```
RollNo: 20-1234
Name:Ahmed
Address: FAST-NU, Block-B, Faisal
Town,Lahore,Pakistan
---
RollNo: 20-1235
Name:Ali
Address: FAST-NU, Block-B, Faisal
Town,Lahore,Pakistan
---
Designation: Assistant Professor
Name:Farooq
Address: FAST-NU, Block-B, Faisal
Town,Lahore,Pakistan
---
0
-1
1
Incompatible types detected that are
not comparable
```

Array

- Basic container built in the language
- Fixed size and type
 - specified at declaration time
 - elements of sub-types of the declared type are allowed
- Implemented as an object
 - special length property to get the size
 - can be substituted for Object
- Multi-dimensional arrays
 - Implemented as array of arrays
 - Jagged arrays possible where size of different arrays may vary

Array with polymorphic invocation

```
import academics.*;

public class Main {

    public static void main(String[] args) {

        Address addr = new Address("FAST-NU, Block-B, Faisal Town",
                                   "Lahore", "Pakistan");

        Person[] persons = new Person[3];
        persons[0] = new Student("20-1234", "Ahmed", addr);
        persons[1] = new Student("20-1235", "Ali", addr);
        persons[2] = new Teacher("Assistant Professor", "Farooq", addr);

        for(Person p : persons){
            p.print();
        }
    }
}
```

Jagged array declaration

```
int[][] array = new int[3][];

for(int i=0; i < 3; i++){
    array[i] = new int[i+1];
    for(int j=0; j < i+1; j++){
        array[i][j] = j;
    }
}
```